EC 69212: Design Lab (Networks)

Topic: Exploring Principles of Congestion in TCP

Week: 10 *Instructor: Amitalok J. Budkuley/Aneek Adhya* *Semester: Autumn 2026*

Lab Plan Overview

- Part I: Baseline Network Exploration
- Part II: Congestion Control Design
- Part III: MultiFlow congestion control
- Part IV: (Challenge) From Congestion control to *Congestion Avoidance*

Prelude

Modern networks must handle multiple senders sharing limited resources. One of the central challenges is *congestion control,* which determines how fast a sender can transmit data without overwhelming the network. In this lab, we simulate a simplified network consisting of a sender, a bottleneck queue, and a receiver. The sender transmits packets based on a congestion window (*cwnd*), which limits the number of packets it can inject into the network during each round-trip time (RTT).

The bottleneck queue accumulates incoming packets and serves them at a fixed rate μ per time unit. The queue has a finite buffer size K, and any packets arriving when the buffer is full are dropped. The evolution of the queue length q can be described by:

$$q_{t+1} = \max\left(q_t + \text{arrivals}_t - \mu, 0\right)$$

where $arrivals_t$ is the number of packets sent in that time step. *Packet drops act as an important signal of congestion in the network.*

A naive sender might simply increase *cwnd* linearly, for example:

$$cwnd \leftarrow cwnd + 1$$

However, this approach often leads to unstable behavior: queues may grow excessively, causing frequent packet drops, and multiple flows may compete unfairly for bandwidth. To address this, TCP employs *feedback-based congestion control strategies.* One common principle is *Linear Increase, Faster-Than-Linear (FTL) Decrease* or LI-FTLD: the sender increases its window slowly when there is no packet loss, and decreases it sharply upon detecting congestion:

Think: 'slow climb when things are okay, quick drop when congestion hits.'

This produces the characteristic *sawtooth pattern* in *cwnd* and stabilizes queue occupancy below the buffer limit.

Multiple Flows: When multiple flows share the same queue, each reacts independently to congestion signals. The principle of *linear increase and faster-than-linear decrease (LI-FTLD)* helps promote fair sharing of bandwidth. Naive strategies, by contrast, may allow one flow to dominate. Through this lab, you will explore the behavior of a naive sender, implement your own congestion control strategies, and finally study LI-FTLD and fairness among multiple competing flows. By experimenting with queues, window updates, and packet loss, you will develop an intuition for how TCP maintains stability, efficiency, and fairness in real networks.

Before starting, consider these questions:

1. What is congestion and how should a sender detect congestion and decide to reduce (or increase) its sending rate?
2. How should the sender safely increase its rate to efficiently use available bandwidth?
3. How do multiple senders sharing a queue affect fairness and stability?
4. How might variability or randomness in packet arrivals impact network behavior?

In a major variation from earlier sessions, in this experiment, we will begin with a simple example code. You will be required to work on this basic skeleton and propose your solutions, and hopefully, discover some fun principles of congestion control.

Optional challenge: If you finish early, you can explore congestion avoidance strategies like RED, which aim to provide early feedback before packet drops occur, smoothing queues and improving fairness.

Buckle up!

==========

Part I: Baseline Network Exploration

1. Aim

Understand congestion in networks by simulating a sender interacting with a bottleneck queue. Observe naive rate increase and its effect on queue and drops.

2. System Model

Sender → Bottleneck Queue → Receiver

Time Abstraction (RTT) We model time in discrete steps, where each step corresponds to one Round-Trip Time (RTT). An RTT represents the time taken for a packet to travel from sender to receiver and for an acknowledgment (ACK) to return.

In this simplified model, feedback about congestion (e.g., packet drops) is assumed to be available at the end of each RTT. Thus, the sender updates its congestion window (`cwnd`) once per time step based on observed network conditions.

At each time step (1 RTT):

- Sender transmits *cwnd* packets.

  Sending Model Simplification: In real TCP, the congestion window (`cwnd`) limits the number of packets that can be "in flight" (i.e., sent but not yet acknowledged). For simplicity, we approximate this by assuming that in each RTT, the sender injects `cwnd` packets into the network.

  Thus, `arrivals = cwnd` represents the number of packets entering the bottleneck queue during that time step.

- Queue accumulates packets
- Link serves μ packets
- Buffer size is limited to K
- Excess packets are dropped

3. Key Variables

- *cwnd*: congestion window
- q: queue size
- μ: service rate
- K: buffer size
- drops: packets dropped

4. Task A: Run the Baseline System

Use the provided MATLAB code:

```
% TCP CONGESTION CONTROL LAB - STARTER CODE

clear; clc; close all;

T = 300;
mu = 50;
K  = 100;

cwnd = 1;
q = 0;

cwnd_trace = zeros(1,T);
q_trace    = zeros(1,T);
drop_trace = zeros(1,T);

for t = 1:T

    arrivals = max(cwnd,0);

    q = q + arrivals;

    served = min(q, mu);
    q = q - served;

    if q > K
        drops = q - K;
        q = K;
    else
        drops = 0;
    end

    loss_event = (drops > 0);

    % BASELINE (no congestion control)
    cwnd = cwnd + 1;

    cwnd = max(cwnd,1);

    cwnd_trace(t) = cwnd;
    q_trace(t)    = q;
    drop_trace(t) = drops;

end

figure;
subplot(3,1,1); plot(cwnd_trace); title('cwnd');
subplot(3,1,2); plot(q_trace); title('Queue');
subplot(3,1,3); plot(drop_trace); title('Drops');
```

Notes:

- **Loss Signal Assumption:** We model congestion detection using an idealized loss signal: if any

packets are dropped during a time step, the sender immediately detects this as a congestion event.

In real TCP, packet loss is inferred indirectly through mechanisms such as duplicate ACKs or timeouts, and may be delayed or noisy. Here, we assume perfect and instantaneous feedback to focus on congestion control behavior.

5. Task B: Explore the Baseline Behavior

- Vary μ and K (e.g., $\mu = 30, 50, 70$, $K = 50, 100, 200$). Observe changes in queue and drops.
- Introduce slight randomness in arrivals: arrivals = max(cwnd + randi([−1,1]),0);
- Record:
  - Maximum and average queue
  - Maximum and average drops
  - Maximum and average *cwnd*

6. Observations and Reflection

Answer:

1. Does the queue stabilize or keep growing? What does *stability* mean to you?
2. How does *cwnd* growth relate to queue behavior?
3. Is increasing the sending rate always beneficial?
4. How do random arrivals affect stability? Comment clearly.

==========

Part II: Congestion Control Design

1. Objective

Modify the system to make it stable and efficient. Explore different congestion window (cwnd) backoff rules.

2. Task C: Think Before Coding

Before coding:

- When should the sender reduce its rate? How much any packet drop, if any, is sufficient?
- How should the sender increase its rate?
- Should increase and decrease be symmetric?
- Propose at least two potential rules for cwnd adjustment.

3. Task D: Implement and Test Own Rules

- Replace the baseline rule: cwnd = cwnd + 1;

Motivation: Observe what happens when the sender is "naive" and ignores congestion. How unstable can the queue become?

- Test at least two different backoff/increase rules, such as:

- Linear decrease on loss, linear increase *Motivation: See if proportional adjustments prevent the queue from growing too fast or dropping too many packets.*
- Multiplicative decrease, linear increase *Motivation: Explore sharper reactions to congestion. Does cutting back more aggressively help stabilize the network?*
- Threshold-based reduction *Motivation: Investigate if reacting to high queue occupancy rather than every drop can reduce oscillations and improve fairness.*

Note: Your goal is not just stability, but also efficiency (high utilization) and fairness.

4. Task E: Implement AIMD

- Implement additive-increase multiplicative-decrease (AIMD):

```
if loss_event
    cwnd = cwnd / 2;
else
    cwnd = cwnd + 1/cwnd;
end
```

- Plot *cwnd*, queue, and drops over time.

How does the AIMD rule you implemented relate to real TCP behavior? Can you identify which TCP variants approximately follow this principle?

Understanding Additive Increase The rule `cwnd = cwnd + 1/cwnd` may appear unusual at first glance. Over one RTT, the sender receives approximately `cwnd` acknowledgments (ACKs), each contributing an increment of `1/cwnd`. Thus, the total increase per RTT is approximately 1 packet.

This models *additive increase*, where the congestion window grows slowly and steadily over time.

5. Observations

Answer:

1. Do you observe oscillatory behavior in *cwnd*?
2. Why does *cwnd* increase slowly but decrease sharply?
3. How does the queue behave now?

==========

Part III: MultiFlowCongestion Control: Test case of two competing flows

6. Task F: Two Competing Flows

In this part, you will explore how two senders share the same bottleneck queue. Each sender maintains its own congestion window, $cwnd_1$ and $cwnd_2$, but both flows feed into a shared queue of size K and service rate μ.

Goals:

- Observe how two flows compete for bandwidth.
- Understand how packet drops affect each flow.
- Explore fairness and stability in a simple network.
- Compare multi-flow behavior with the single-flow case.

Modeling Drop Allocation Across Flows When multiple flows share a queue, packet drops must be attributed to individual flows. A simple approach is to assign drops probabilistically based on each flow's contribution to total arrivals.

For example, if Flow 1 contributes a fraction p_1 of the total arrivals, then it experiences a loss event with probability p_1 when drops occur. Note that both flows observe congestion only through the shared queue, and do not directly coordinate with each other. This is a classic example of *decentralized design.*

Suggested MATLAB Skeleton:

```
% Two-flow shared queue setup
T = 300;        % number of time steps
mu = 50;        % link service rate
K  = 100;       % buffer size

% Initialize congestion windows
cwnd1 = 1;
cwnd2 = 1;

% Initialize queue
q = 0;

% Trace variables for plotting
cwnd1_trace = zeros(1,T);
cwnd2_trace = zeros(1,T);
q_trace     = zeros(1,T);
drop_trace  = zeros(1,T);

for t = 1:T

    % --- Packet arrivals ---
    arrivals1 = max(cwnd1,0);
    arrivals2 = max(cwnd2,0);
    total_arrivals = arrivals1 + arrivals2;

    % --- Update shared queue ---
    q = q + total_arrivals;
```

```
    served = min(q, mu);
    q = q - served;

    % --- Packet drops ---
    if q > K
        drops = q - K;
        q = K;
    else
        drops = 0;
    end

    % --- Decide how drops affect each flow ---
    % Option 1: proportional to contribution
    % Option 2: both reduce equally
    % Option 3: random
    % (Implement your choice here)

    % Example: proportional drops based on contribution

if total_arrivals > 0
    p1 = arrivals1 / total_arrivals;
    p2 = arrivals2 / total_arrivals;
else
    p1 = 0;
    p2 = 0;
end

loss_event1 = (drops > 0) && (rand < p1);
loss_event2 = (drops > 0) && (rand < p2);
%% TODO: implement your own cwnd update rules
    %loss_event1 = ... % define based on your drop rule
    %loss_event2 = ... % define based on your drop rule

    % --- Update congestion windows ---
    % Example placeholder (replace with your own rule)
    if loss_event1
        cwnd1 = max(1, cwnd1 - 1);
    else
        cwnd1 = cwnd1 + 0.5;
    end
    if loss_event2
        cwnd2 = max(1, cwnd2 - 1);
    else
        cwnd2 = cwnd2 + 0.5;
    end

    % --- Save traces ---
    cwnd1_trace(t) = cwnd1;
    cwnd2_trace(t) = cwnd2;
    q_trace(t)     = q;
    drop_trace(t)  = drops;

end
```

```
% ---- Plot results ----
figure;
plot(cwnd1_trace, 'r', 'LineWidth',1.5); hold on;
plot(cwnd2_trace, 'b', 'LineWidth',1.5);
xlabel('Time'); ylabel('cwnd');
title('Two-Flow Congestion Window Evolution');
legend('Flow 1','Flow 2');
grid on;
```

Observation Guidelines:

- Compare the evolution of $cwnd_1$ and $cwnd_2$. Are both flows getting roughly equal bandwidth?
- Do the flows reduce their rates at the same time, or do they desynchronize?
- How does the shared queue behave compared to the single-flow case?
- Try different drop-allocation rules and observe the impact on fairness (e.g., average *cwnd* ratio).

Motivation: This exercise demonstrates the fundamental principle of feedback and interaction in networks: multiple flows respond independently to congestion signals, and simple local rules can lead to complex global behaviors like fairness, oscillation, and stability.

Measuring Fairness To quantify how fairly bandwidth is shared between flows, you may use simple metrics such as:

- Ratio of average congestion windows: $\frac{\overline{cwnd_1}}{\overline{cwnd_2}}$
- Difference in average throughput between flows

Optionally, you can compute Jain's Fairness Index:

$$J = \frac{(x_1 + x_2)^2}{2(x_1^2 + x_2^2)}$$

where x_1 and x_2 represent the average rates (or congestion windows) of the two flows. A value close to 1 indicates fair sharing.

7. Reflection

Answer:

1. Why does TCP exhibit sawtooth behavior?
2. What role does packet loss play in stability and fairness?
3. How do different backoff rules impact queue occupancy?

Attempt one or more of the following variations.

- Introduce stochastic packet loss: randomly drop 5–10% of packets
- Explore probabilistic backoff: e.g., reduce cwnd by half only 50% of the time
- Introduce asymmetric link capacities: one sender has $\mu_1 \neq \mu_2$
- Compare throughput, fairness, and queue behavior

Key Insights

- TCP stability emerges from local feedback rules interacting with the network
- Sawtooth behavior reflects additive increase and multiplicative decrease
- Fairness is not guaranteed with naive rules; AIMD provides predictable, fair allocation
- Design exploration helps understand trade-offs between efficiency, stability, and fairness

==========

Part IV: (SuperMotivationChallenge) From *congestion control* to *congestion avoidance*..Gimme RED!

From Congestion Control to Congestion Avoidance

So far, you have explored how senders react to packet drops using simple congestion control rules. In practice, reacting only after packets are dropped can lead to buffer overflows, oscillations, and inefficient network utilization.

Congestion avoidance aims to detect early signs of congestion before the queue fills up. The principle is simple: reduce sending rate or slow growth before losses occur, rather than waiting for packet drops. One common approach is Random Early Detection (RED), which probabilistically drops packets or signals congestion as the queue length approaches a pre-defined threshold.

The goal of this extension is to simulate early congestion feedback and observe how the system behaves differently from the drop-only schemes explored earlier.

Optional MATLAB Simulation: Congestion Avoidance

- Introduce a queue threshold Q_{th} (e.g., 70–80% of buffer).
- If $q > Q_{th}$, trigger a probabilistic backoff or reduce cwnd slightly (instead of waiting for full drops).

```
if q > Q_th
    if rand < 0.5
        cwnd = cwnd * 0.9; % reduce slightly
    end
end
```

- Compare to the drop-based rules you implemented in Part II:
  - How does the queue behave?
  - Are packet drops reduced?
  - How does fairness between flows change?

Tip: Track and plot cwnd and queue over time for each flow, and compare with earlier experiments.